

Golang 核心知识点清单

从零基础到进阶 · 系统化语言知识脉络

一、基础语法

一、基础语法

1.1 变量与常量声明

var 关键字声明：使用 `var` 关键字声明变量，形如 “var 名称 类型 = 值”，类型可省略由编译器自动推断。

- **短变量声明：**在函数内部使用 `:=` 声明并初始化变量，类型自动推断，是最常用的写法。
- **批量声明：**使用 `var (...)` 语法一次性声明多个变量，使声明集中、整洁。
- **常量声明：**使用 `const` 声明不可修改的值，支持 `iota` 常量生成器批量递增赋值。

1.2 基本数据类型

- **整型：**包括有符号 `int`、`int8/16/32/64` 与无符号 `uint`、`uint8/16/32/64` 以及 `uintptr`。
- **浮点型：**`float32` 与 `float64` 用于存储带小数的数值，运算结果可能产生精度误差。
- **布尔型：**`bool` 类型只有 `true` 与 `false` 两个取值，不能与数值互转。
- **字符串：**`string` 是不可变的 UTF-8 字节序列，可用双引号或反引号表示。
- **零值：**未显式初始化的变量自动获得其类型的零值，如数值为 0、字符串为 ""、布尔为 false。
- **显式类型转换：**Go 不支持隐式转换，需用 `T(v)` 形式显式转换，保证类型安全。

1.3 控制结构

- **if / else：**条件分支结构，条件表达式无需括号，且支持在条件前加简短初始化语句。
- **for 循环：**Go 中唯一的循环语句，可表达经典三段式、while 式与无限循环三种形式。
- **switch：**多分支选择结构，默认不贯穿（无需 break），支持无表达式的条件 switch。
- **select：**在多个通道操作中选择一个可执行的分支执行，是并发编程的核心控制结构。

二、函数与指针

二、函数与指针

2.1 函数定义与多返回值

- **函数定义**：使用 `func` 关键字定义函数，参数与返回值均需显式声明类型。
- **多返回值**：函数可同时返回多个值，最典型的是“结果 + error”的组合。
- **命名返回值**：为返回值命名后可在函数体内直接赋值，并简化 `return` 语句。
- **可变参数**：使用 `...T` 接收不定数量的参数，在函数内部以切片形式访问。

2.2 匿名函数与闭包

- **匿名函数**：没有名字的函数，可赋值给变量、作为参数传递或立即执行。
- **闭包**：函数可捕获并引用其外部作用域的变量，即使外部函数已返回仍可访问。

2.3 defer 延迟调用

- **defer 机制**：将函数调用推迟到外层函数返回前执行，常用于资源释放与解锁。
- **执行顺序**：多个 `defer` 按后进先出（LIFO）顺序执行，即最后 `defer` 的最先执行。

2.4 指针

- **指针概念**：指针存储另一变量的内存地址，用 `*T` 表示类型、`&` 取地址。
- **解引用**：使用 `*p` 读取或修改指针所指向变量的值。
- **使用场景**：在函数间传递大对象、修改外部变量，以及表达“可为空”的引用语义。
- **new 分配**：使用 `new(T)` 分配内存并返回指向该类型零值的指针。

三、核心数据结构

3.1 数组与切片

- **数组 (Array)**：长度固定且同类型的元素集合，数组长度是其类型的一部分。
- **切片 (Slice)**：长度可变的动态视图，由指向底层数组的指针、长度与容量三部分组成。
- **切片创建**：通过 `make`、字面量或对数组 / 切片进行切片操作来创建。
- **append 追加**：向切片末尾追加元素，容量不足时自动扩容并返回新切片。
- **copy 复制**：在两个切片之间复制元素，复制长度为两者中的较小值。
- **两者区别**：数组长度固定且为值类型（赋值即拷贝），切片长度可变且共享底层数组。

3.2 映射 (Map)

- **定义**：map 是无序的键值对集合，使用 `map[K]V` 声明键与值的类型。
- **创建**：使用 `make` 或字面量初始化 map，未初始化的 map 为 `nil`。
- **增删改查**：通过下标读写元素、用 `delete` 删除；读取时用第二个返回值判断键是否存在。
- **遍历**：使用 `for range` 遍历键值对，遍历顺序不保证固定。

四、类型与面向对象抽象

4.1 结构体与方法

- **结构体 (Struct)**：用 `type` 与 `struct` 定义复合类型，将多个相关字段聚合到一起。
- **字段访问**：使用点号 `.` 访问或修改结构体字段，也可取字段地址赋值。
- **方法 (Method)**：为自定义类型绑定函数，接收者可声明为值类型或指针类型。
- **指针接收者**：使用指针接收者可修改接收者本身并避免大对象拷贝，是更常用的写法。

4.2 接口

- **接口定义**：接口声明一组方法签名，描述“能做什么”而不涉及具体实现。
- **隐式实现**：类型只要实现了接口的全部方法即自动满足该接口，无需显式声明。
- **空接口**：`interface{}` 可持有任意类型的值，是泛型出现前的通用容器。

4.3 类型断言

- **类型断言**：使用 `x.(T)` 将接口值断言为具体类型，失败时会触发 panic。
- **安全断言**：使用 `v, ok := x.(T)` 判断断言是否成功，避免直接 panic。
- **type switch**：用 switch 配合类型断言，按动态类型对接口值分情况处理。

五、并发编程基础

5.1 协程 (Goroutine)

- **启动方式**：在函数调用前加 `go` 关键字即可创建一个并发执行的 goroutine。
- **轻量特性**：goroutine 由 Go 运行时调度，栈可动态增长，开销远小于操作系统线程。

5.2 通道 (Channel)

- **通道概念**：channel 是 goroutine 之间通信的管道，使用 `make(chan T)` 创建。
- **无缓冲通道**：发送与接收必须同步配对，天然提供同步语义。
- **有缓冲通道**：可缓存一定数量的元素，缓冲区未满 / 非空时发送 / 接收不阻塞。
- **关闭通道**：使用 `close` 关闭通道，接收方可据此判断通道已关闭。
- **通道方向**：可为通道参数指定只发送或只接收方向，即 `chan<-` 与 `<-chan`。

5.3 基本并发控制

- **sync.WaitGroup**：用于等待一组 goroutine 完成，通过 Add / Done / Wait 协调。
- **sync.Mutex**：互斥锁，通过 Lock / Unlock 保护共享资源，避免数据竞争。
- **数据竞争**：多个 goroutine 并发读写同一变量且未同步时，会产生不确定的结果。

六、错误处理

6.1 Error 接口

- **error 接口**: 内置接口仅包含 `Error() string` 一个方法，任何实现它的类型都是错误。
- **创建错误**: 使用 `errors.New` 或 `fmt.Errorf` 创建错误值。
- **错误传播**: 函数通过多返回值将错误返回给调用者，由调用者判断并处理。
- **错误包装**: 使用 `fmt.Errorf` 的 `%w` 包装底层错误，`errors.Is / As` 用于解包判断。

6.2 Panic 与 Recover

- **panic**: 表示程序无法继续运行的严重错误，触发后沿调用栈向上传播。
- **recover**: 仅在 `defer` 中调用可捕获 `panic` 并让程序恢复执行。
- **使用原则**: `panic` 用于不可恢复的致命错误，常规错误应通过返回值处理。

七、高级特性与工程化

7.1 泛型 (Generics)

- **泛型概念**: 使用类型参数让函数与类型可复用，Go 1.18 起支持。
- **类型约束**: 通过 `interface` 定义类型参数必须满足的约束，如 `any`、`comparable`。

7.2 反射 (Reflection)

- **reflect 包**: 提供在运行时检查类型与值的机制。
- **常用操作**: `reflect.TypeOf` 获取类型，`reflect.ValueOf` 获取并操作值。
- **使用场景**: 用于序列化、通用框架等需处理未知类型的情况，但性能较低应谨慎使用。

7.3 Go Modules 包管理

- **go mod init**: 初始化模块并生成 `go.mod` 文件，声明模块路径。
- **依赖管理**: `go get` 添加依赖，`go mod tidy` 清理并同步依赖。
- **go.sum 校验**: 记录依赖的校验和，保证构建可复现、可验证。

7.4 常用标准库

- **fmt**: 格式化输入输出，如 `Printf`、`Sprintf`、`Scanf`。
- **io**: 定义 `Reader / Writer` 等基本 I/O 接口，是流式处理的基础。
- **net/http**: 构建 HTTP 客户端与服务端，用于 Web 服务开发。
- **encoding/json**: 在 JSON 与 Go 数据结构之间进行序列化与反序列化。

7.5 编写测试

- **testing 包**：内置测试框架，测试函数以 `Test` 开头并接收 `*testing.T`。
- **go test 命令**：运行测试，支持单元测试、基准测试与示例测试。
- **表驱动测试**：将多组输入输出放入表格循环执行，是 Go 社区推荐的测试风格。